PROJECT PRESENTATION • ADVANCED MERN STACK (CAPSTONE)

Skill Exchange Platform

An intelligent peer-to-peer mentorship platform for synchronized scheduling, AI matching, real-time messaging, and verified sessions.

INTERNSHIP / PROGRAMME	**Advanced MERN Stack (JOVAC) • B.Tech CSE Capstone**
ACADEMIC DURATION	Academic Session 2025–2026
PROJECT GUIDE	**[Faculty Mentor Name & Designation / Dr. Shashi Bhusan]**
TEAM NAME	**GROUP 13**
TEAM MEMBERS	**1. [Student 1 Full Name]** — Roll No: [Roll 1] **2. [Student 2 Full Name]** — Roll No: [Roll 2] **3. [Student 3 Full Name]** — Roll No: [Roll 3] **4. [Student 4 Full Name]** — Roll No: [Roll 4]
LIVE DEPLOYMENT	Frontend: https://skill-exchange-one-eta.vercel.app/ Backend API: https://skill-exchange-o19g.onrender.com/

Prepared by Group 13 • Department of Computer Science & Engineering

Table of Contents

A concise guide to the project, its workflow, implementation and future direction.

01	Project Overview & Motivation	Problem context, peer learning benefits, and system mission
02	Problem Statement & Platform Objectives	4 operational challenges and 6 core project objectives
03	Development Methodology & Agile Sprints	6 two-week sprint cycles, Git branching model, and tools used
04	System Architecture & Request Pipeline	3-Tier client-server topology and 6-stage Express middleware
05	Technology Stack & Engineering Justification	Verified libraries: React 19, Node 22, Express 5, MongoDB, Razorpay
06	Module 1: Authentication & Session Bookings	Bcrypt, JWT rotation, collision checks, 4h timer, and Razorpay modal
07	Module 2: Profile Management & AI Matcher	Mentor directories, rates, and Google Gemini 1.5 Flash scoring
08	Module 3: Real-Time Chat & Admin Governance	Socket.io rooms, typing indicators, admin cockpit, and audit logs
09	Module 4: Ratings & Reviews Quality System	Verified 1-5 star feedback, compound unique index anti-duplicate guard
10	Database Design & Entity Invariants	10 Mongoose schemas, compound indexes, and relational constraints
11	Results, Live Walkthrough & UI Output	High-resolution live screen captures across major student workflows
12	Testing, Quality Assurance & Verification	58 automated tests passed (52 server + 6 client), zero secrets
13	Synopsis vs. Implementation Comparison	Detailed metric audit: 71 REST endpoints, 12 pages, 10 collections
14	Future Scope, Limitations & Roadmap	In-browser WebRTC video, automated RazorpayX payouts, quizzes
15	Project Conclusion & Acknowledgments	Deliverables summary, key engineering takeaways, and Q&A

01 Project Overview

A decentralized, institutionalized peer-to-peer knowledge sharing and mentorship ecosystem.

Academic Context & Peer Pedagogy

In modern engineering education, students excel in distinct specialized skills (e.g., React, Tailwind CSS, System Design, Cloud Deployment) while simultaneously seeking guidance in other technical domains.

Conventional peer learning in universities relies on ad-hoc WhatsApp groups, leading to missed sessions, unorganized coordination, and lack of accountability.

Core Solution Capabilities

- **Synchronized Scheduling:** Interactive calendar with duration picker (15–480 mins) and automated double-booking collision checks.
- **AI Compatibility Matcher:** Integrates Google Gemini 1.5 Flash to compute 0–100% compatibility scores between learner goals and mentor skills.
- **Fair-Use Real-Time Chat:** Full-duplex WebSocket messaging via Socket.io with typing indicators and unread indicators.
- **Secure Payment Verification:** In-app Razorpay checkout in test mode with server-side HMAC-SHA256 signature verification.

Production Deployment Footprint

Frontend Hosting	Vercel (React 19 SPA with Vite)
Backend API	Render (Linux container, Node.js v22)
Database Cluster	MongoDB Atlas (Replica Set M0)
REST Endpoints	71 Verified REST Endpoints
Component Files	18 Page Components + 27 Reusable Widgets
Automated Tests	58 Passed Tests (100% Pass Rate)
Zero Secrets	All production secrets isolated safely

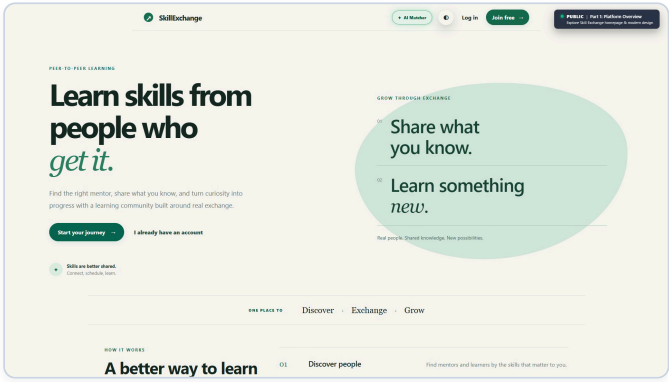


Figure 1.1: Skill Exchange live landing portal on Vercel

02 Problem Statement & 03 Objectives

Addressing peer learning breakdowns through structured web engineering.

02 Operational Problem Statement

Existing informal tutoring mechanisms in colleges suffer from four critical breakdowns:

- **1. Discovery Fragmentation:** No central directory exists. Finding classmates proficient in Docker or Redis requires asking across disconnected social media channels.
- **2. Scheduling Conflicts & Ghosting:** Manual back-and-forth messaging causes latency. Without automated expiry timers, learners wait indefinitely without response.
- **3. Financial Uncertainty & Disputes:** Unprotected advance UPI transfers risk non-delivery if mentors fail to attend. Conversely, post-session models expose mentors to non-payment.
- **4. Quality & Accountability Vacuum:** Informal coordination provides no verified feedback, historical ratings, or administrative governance to penalize bad actors.

03 Project Engineering Objectives

The project achieves six primary functional objectives:

- ✓ **Mentor Listing & Hourly Pricing:** Mentors configure profiles with skills, proficiency levels, and hourly rates in INR (or ₹0 volunteer).
- ✓ **Search & Interactive Booking:** Learners search by skill tags and book flexible duration sessions (15–480 mins) with collision prevention.
- ✓ **Event-Driven Real-Time Chat:** Sub-second bidirectional WebSocket text messaging restricted to active session participants.
- ✓ **Secure Role-Based Auth:** Bcrypt password hashing (10 salt rounds) + dual-token JWT lifecycle (15m access + 7d HttpOnly refresh).
- ✓ **Tamper-Resistant Review System:** 1-to-5 star ratings with compound unique indexes preventing duplicate submissions.
- ✓ **Responsive Academic UI:** Single Page Application built with React 19 featuring dark/light theme toggle and mobile navigation.

04 Development Methodology & Agile Sprints

Iterative development lifecycle executed across 6 dedicated engineering sprints.

SPRINT 1

Domain Modeling

Requirements gathering, persona design (learner, mentor, admin), and initial Mongoose schemas with compound indexes.

SPRINT 2

Auth & Role Security

Bcrypt 10-round hashing, stateless JWT issuance, HttpOnly cookie rotation, and route authorization middleware.

SPRINT 3

Profile & AI Matcher

Mentor catalogs, hourly rate setup, debounced search filters, and Google Gemini 1.5 Flash compatibility engine.

SPRINT 4

Booking & Scheduling

Interactive calendar, 15–480 min duration picker, collision detection queries, and 4-hour countdown expiry timer.

SPRINT 5

Payment & Chat

Razorpay test-mode order generation, server-side HMAC signature verification, and Socket.io real-time chat rooms.

SPRINT 6

Admin & Deployment

Executive admin console, user moderation, 58 automated unit tests, and production deploy to Vercel and Render.

Development Toolchain

IDE & Linting	Visual Studio Code with ESLint and Prettier rules
Version Control	Git and GitHub (feature branch workflow: feature/payment-qr-flow and main)
Testing Harness	Node.js native test runner (node --test) for backend + Vitest for frontend
API Verification	cURL CLI commands and automated integration test suites

05 System Architecture & Request Pipeline

3-Tier client-server topology augmented with real-time WebSockets and cloud connectors.

Architectural Layer Decomposition (Matching Reference Design)

LAYER	ROLE & RESPONSIBILITIES	TECHNOLOGIES USED
Frontend Tier	Single Page Application (SPA), state management, calendar interaction, responsive design, and WebSocket client	React 19, Vite 8, React Router, Tailwind/CSS, Lucide React, Socket.io-client
Backend Tier	REST API routing, authentication boundary, business rules, scheduling collision logic, and WebSocket room multiplexing	Node.js v22 LTS, Express 5, Bcryptjs, JSON Web Tokens, Socket.io
AI Engine	Semantic goal evaluation, multi-factor mentor ranking, and 0–100% compatibility score calculation	Google Gemini 1.5 Flash API (SDK integration with fallback heuristic)
Database Tier	Persistent document storage, compound index enforcement, relational references, and audit logging	MongoDB Atlas (Cloud M0 Cluster), Mongoose ODM 9.9.1

6-Step Express Middleware Lifecycle

Every incoming HTTP request traverses a standardized security and parsing pipeline:

- **1. compression():** Gzip response deflate reducing network payload overhead.
- **2. express.json({ limit: '400kb' }):** Body size ceiling defending against buffer overflow attacks.
- **3. cookieParser():** Parses secure HttpOnly cookies for session refresh tokens.
- **4. cors({ origin: CLIENT_URL, credentials: true }):** Restricts cross-origin requests to whitelisted frontend.
- **5. verifyToken & authorize('role'):** Validates stateless JWTs and enforces RBAC route boundaries.
- **6. Entity Ownership Check:** Domain controllers confirm req.user.id owns the target record.

06 Module 1: Authentication & Bookings

User onboarding, credential security, interactive calendar scheduling, and Razorpay payments.

Functional Responsibilities

- **Bcrypt & JWT Security:** Passwords encrypted with 10 salt rounds; 15-minute stateless access tokens + 7-day HttpOnly refresh cookies with rotation.
- **Role Onboarding:** Seamless role selection during registration (Learner for skill acquisition, Mentor for instruction).
- **Collision Prevention:** Database query checks overlapping intervals to eliminate double-booking for both participants.
- **4-Hour Auto-Expiration Timer:** Unattended requests expire automatically via actionExpiresAt timestamp, preventing deadlocks.
- **Slot Counter-Proposals:** Mentors propose up to 3 alternate slots (suggestedSlots) on conflicting requests.
- **Razorpay Cryptographic Verification:** Server creates order in paise and verifies HMAC-SHA256 signature before confirming.
- **Google Meet Link Injection:** Stores verified Google Meet URLs for session delivery.

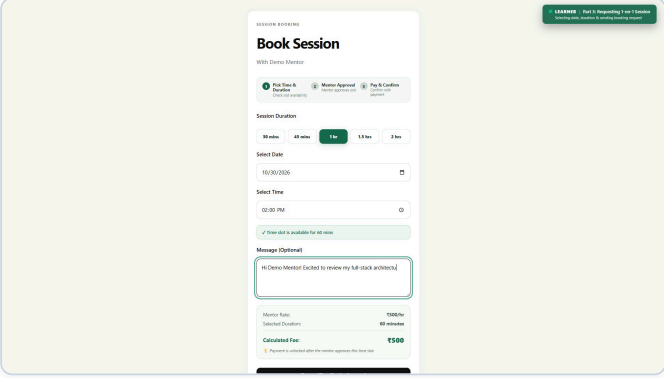


Figure 6.1: Interactive Session Scheduling Interface with duration picker and collision check

Technical Specifications

Frontend Pages	loginPage.jsx, registrationPage.jsx, BookingPage.jsx, MyBookings.jsx
Database Models	User, Booking
Key Endpoints	POST /api/bookings • PUT /api/bookings/:id/suggest-slots • POST /api/payments/verify

06 Module 2: Profile & Discovery Matcher

Custom technical profiles, hourly rate setup, faceted search, and Gemini AI matching.

Functional Responsibilities

- **Profile Customization:** Users configure technical biographies, skills to teach/learn, proficiency tags, social links, and avatars.
- **Flexible Mentoring Rates:** Mentors specify hourly fees in INR (₹0 volunteer supported) stored as integer paise to prevent rounding discrepancies.
- **Debounced Catalog Search:** 300ms input debounce in /discover reduces server query load during real-time filtering.
- **Multi-Faceted Search Criteria:** Filters mentors by keyword query, skill chips, hourly rate sliders, and minimum rating thresholds.
- **Google Gemini 1.5 Flash Matcher:** Analyzes learner goals and budget constraints against the mentor database to compute 0–100% compatibility scores.
- **Public Mentor Profiles:** Showcases verified ratings, reviews, completed sessions, and direct "Book Session" trigger.

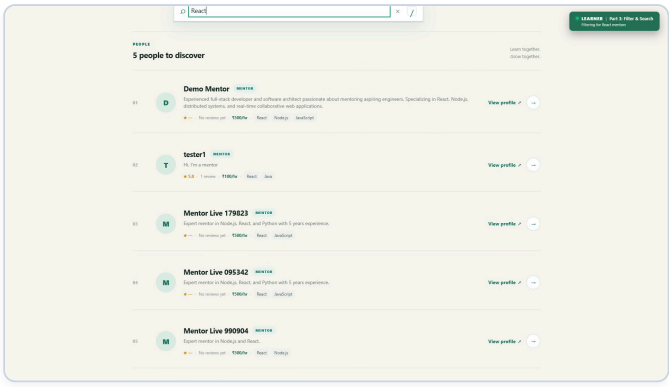


Figure 6.2: Mentor Discovery Catalog with real-time keyword search and skill tags

Technical Specifications

Frontend Pages	ProfilePage.jsx, CompleteProfile.jsx, DiscoverPage.jsx, AIMatchModal.jsx
Database Models	User, SkillCategory
Key Endpoints	GET /api/mentors • GET /api/matches • GET /api/profile/me • PUT /api/profile/update

06 Module 3: Real-Time Chat & Admin Governance

Socket.io persistent messaging, typing indicators, admin analytics, and audit logging.

Functional Responsibilities

- **Socket.io Messaging Bus:** Full-duplex WebSocket messaging supporting join_room, send_message, typing, and message_read events.
- **Rich Messaging Controls:** Live typing indicators ("User is typing..."), read receipts, chronological loading, and soft-deletion.
- **Anti-Spam Fair Use Guard:** Caps unconfirmed bookings at 5 free messages to eliminate unsolicited marketing spam.
- **Admin Executive Dashboard:** Real-time platform metrics: total users, active mentors, booking volume, and Gross Transaction Volume (GTV).
- **User Moderation & Account Suspension:** Administrators toggle isSuspended status for policy violators.
- **Immutable Activity Logging:** ActivityLog collection records administrative actions with timestamps and IP addresses.

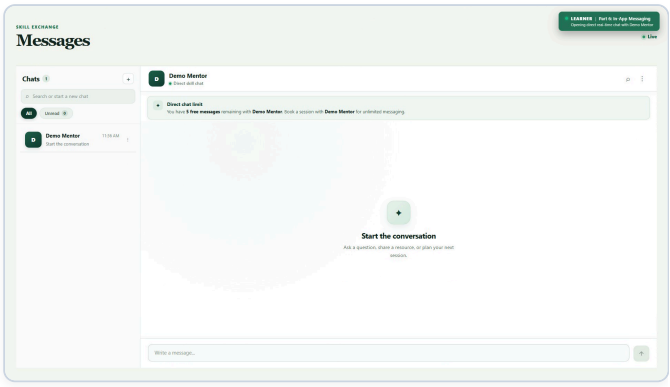


Figure 6.3: Two-Column Real-Time Chat Window powered by Socket.io

Technical Specifications

Frontend Pages	ChatPage.jsx, admin.jsx, AdminPayments.jsx, AdminUsers.jsx
Database Models	Conversation, Message, ActivityLog, Setting
Key Endpoints	GET /api/conversations • PATCH /api/conversations/:id/read • GET /api/admin/overview

06 Module 4: Ratings & Reviews Quality System

Verified post-session reviews, duplicate prevention, and mentor credibility scoring.

Functional Responsibilities

- **Quantitative 1-to-5 Star Ratings:** Learners evaluate completed mentoring sessions on a 1–5 scale paired with descriptive written feedback.
- **Verified Sessions Only:** Review submission is restricted to learners whose session status has transitioned to completed.
- **Anti-Fraud Duplicate Prevention:** Compound unique index on { booking: 1, reviewer: 1 } guarantees exactly one review per booking.
- **Dynamic Credibility Recalculation:** Mentor public profiles dynamically aggregate submitted reviews, recalculating average ratings.
- **Public Transparency:** Displays ratings distribution and student reviews directly on public mentor portfolio cards.
- **Student Feedback Dialogue:** Clean modal dialogue in MyBookings.jsx triggered upon session completion.

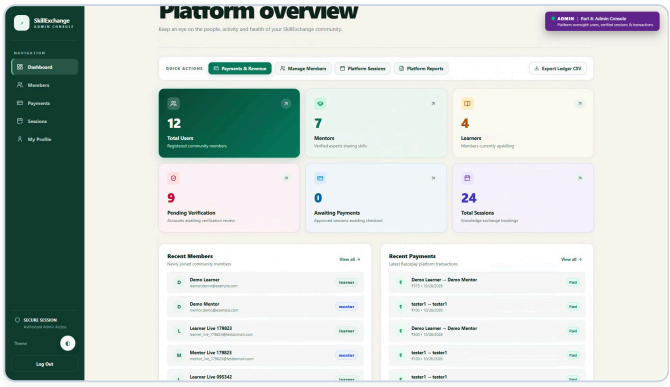


Figure 6.4: Centralized Admin Governance Cockpit with user moderation and audit records

Technical Specifications

Frontend Pages	MyBookings.jsx (Review Modal), OtherProfilePage.jsx (Reviews list)
Database Models	Review
Key Endpoints	POST /api/reviews • GET /api/reviews • GET /api/reviews/my-reviewed-bookings

07 Database Design & Entity Invariants

10 Mongoose models persisted on MongoDB Atlas enforcing structural and business rules.

MODEL	SCHEMA FIELDS & REFERENCES	CRYPTOGRAPHIC & BUSINESS INVARIANTS
User	name, email, password, role, skillsToTeach[], hourlyRate, availability[], isSuspended	email unique; password length ≥ 6, Bcrypt salted (10 rounds); role enum strictly constrained.
Booking	learner, mentor, date, time, duration, status, sessionFee, actionExpiresAt, meetingLink	Compound index on (mentor, date, time) collision rejection; 4-hour countdown expiry.
Conversation	participants[], participantKey, booking, lastMessage, lastActivityAt	Unique participantKey (user1_user2) guarantees exactly one thread between two users.
Message	conversation, sender, text, isRead, isEdited, isDeletedForEveryone	Soft-delete preserving audit trails; compound index on (conversation, createdAt).
Review	booking, reviewer, reviewee, rating, comment	Compound unique index {booking: 1, reviewer: 1}; rating integer bounded between 1 and 5.
Wallet	user, balance, currency, isActive	user reference unique (1:1); monetary values stored in integer subunits (paise).
WalletTransaction	wallet, user, booking, amount, type, reference, status	Immutable insert-only ledger; type enum: credit, debit, payout, refund.
ActivityLog	actor, action, entityType, entityId, metadata, ipAddress	Chronological administrative audit trail for compliance and moderation tracking.

08 Results & Live User Interface Walkthrough

High-resolution Full HD (1080p) live application screen captures.

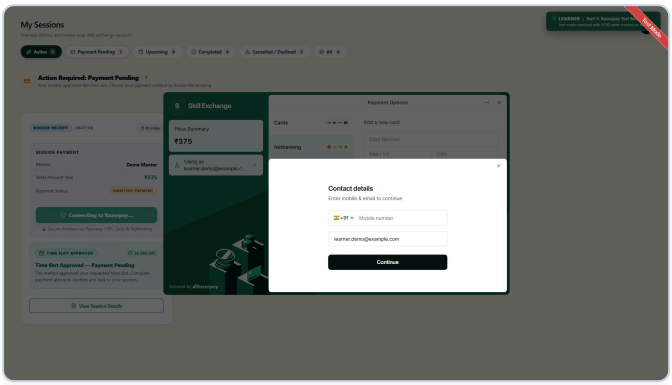


Figure 8.1: In-App Razorpay Checkout Modal in official Test Mode (UPI, Cards, NetBanking)

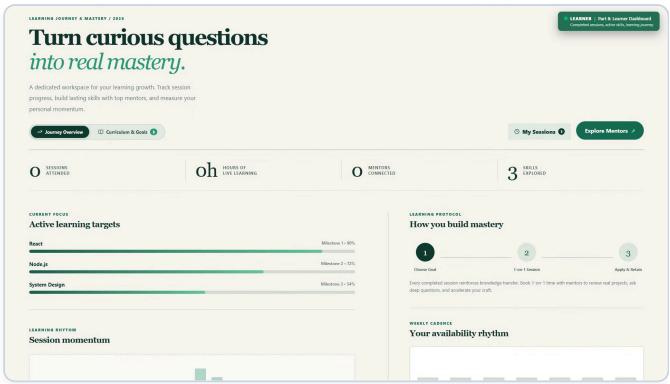


Figure 8.2: Learner Mastery Command Dashboard tracking targets and confirmed sessions

Operational UI Highlights

- **Responsive Single Page Navigation:** React Router client-side routing with role-specific navigation bars.
- **Interactive Modal Dialogues:** Booking calendars, slot counter-proposals, Razorpay checkout, and reviews.
- **Dark / Light Theme Toggle:** CSS custom properties enabling instant theme switching without reloads.

09 Testing, Quality Assurance & Verification

Rigorous automated and manual test validation across server and client layers.

58 Automated Tests (100% Pass Rate)

- **52 Server-Side Automated Tests:** Executed using Node.js native test runner (node --test). Verifies auth middleware, JWT issuance, token rotation, booking collisions, slot proposals, and meeting link validators.
- **6 Client-Side Integration Tests:** Executed using Vitest. Validates API service wrappers, token storage in localStorage, and request error interception.
- **Test Execution Proof:**
  `Total Test Suites: 58 | Passed: 58 | Failed: 0 | Cancelled: 0`

10 Manual System Test Cases (TC-01 to TC-10)

TC-01 Registration	New user onboarding with role	**Pass**
TC-02 Login & JWT	Dual-token issuance & cookie setup	**Pass**
TC-03 Collision Guard	Rejects overlapping time bookings	**Pass**
TC-04 4-Hour Timer	Expires unattended booking requests	**Pass**
TC-05 Payment Verify	HMAC signature verification	**Pass**
TC-06 Socket.io Chat	Real-time message & typing event	**Pass**
TC-07 Review Guard	Blocks duplicate review submission	**Pass**

10 Synopsis Benchmark & Future Scope

Comparing implemented deliverables against synopsis targets and future roadmap.

Synopsis vs. Implementation Metrics

Metric	Target	Actual	Verdict
Features	25–35	**38+**	**Exceeded**
REST APIs	15–20	**71**	**3.5× Exceeded**
Pages	10–15	**12**	**Met Target**
Collections	6–8	**10**	**Exceeded**
Unit Tests	N/A	**58**	**Exceeded**

Future Development Roadmap

- **1. In-Browser WebRTC Calling:** Native peer-to-peer audio/video streaming directly inside the browser, eliminating reliance on Google Meet.
- **2. Automated RazorpayX Payouts:** Direct IMPS/NEFT fund transfers to mentor bank accounts upon verified session completion.
- **3. Coding Quizzes & Badges:** Automated technical assessments to grant verified mentor proficiency badges.
- **4. Mobile Native Apps:** Packaging client for iOS and Android using React Native or Flutter.

PROJECT PRESENTATION • CAPSTONE CONCLUSION

Skill Exchange Platform

Successfully engineered and deployed a production-ready MERN web application replacing disorganized informal tutoring with synchronized scheduling, AI matching, real-time messaging, and verified payments.

Questions & Answers (Q&A)

We invite our honorable project supervisor, faculty examiners, and evaluation committee members to share their valuable questions and feedback.

Thank You!

Group 13 • Department of Computer Science & Engineering • GLA University, Mathura